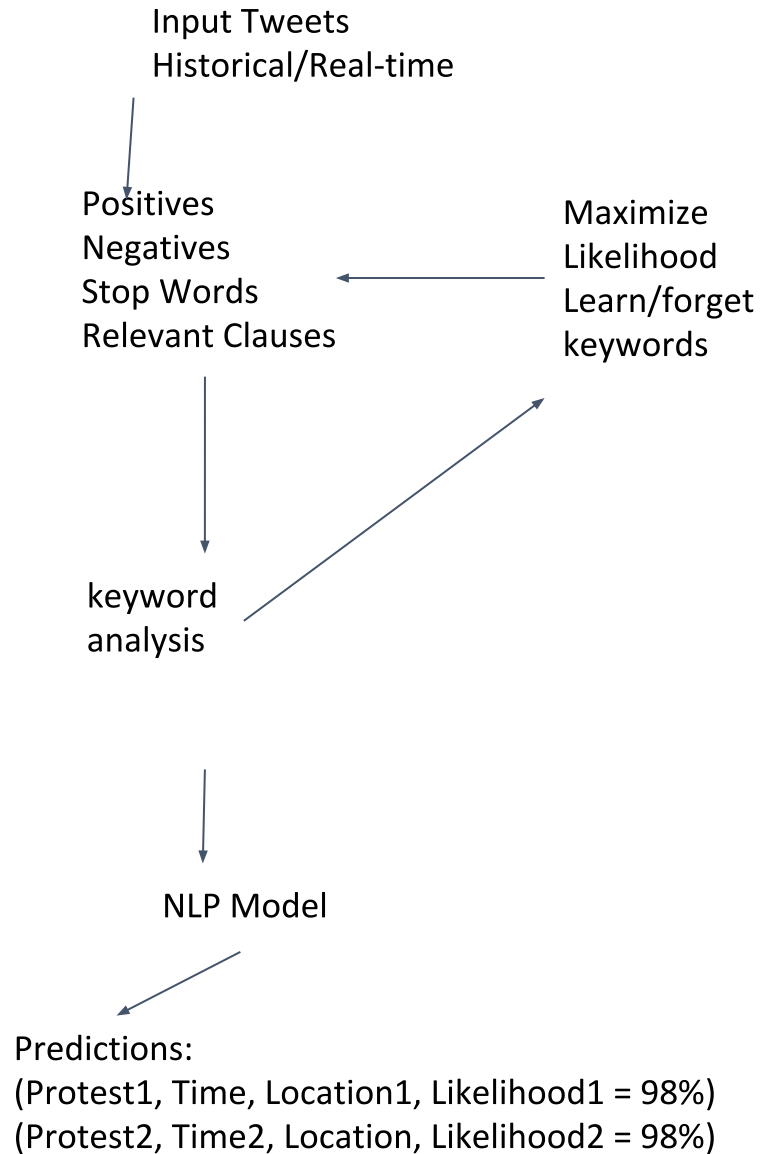


Twitter Protest Filter



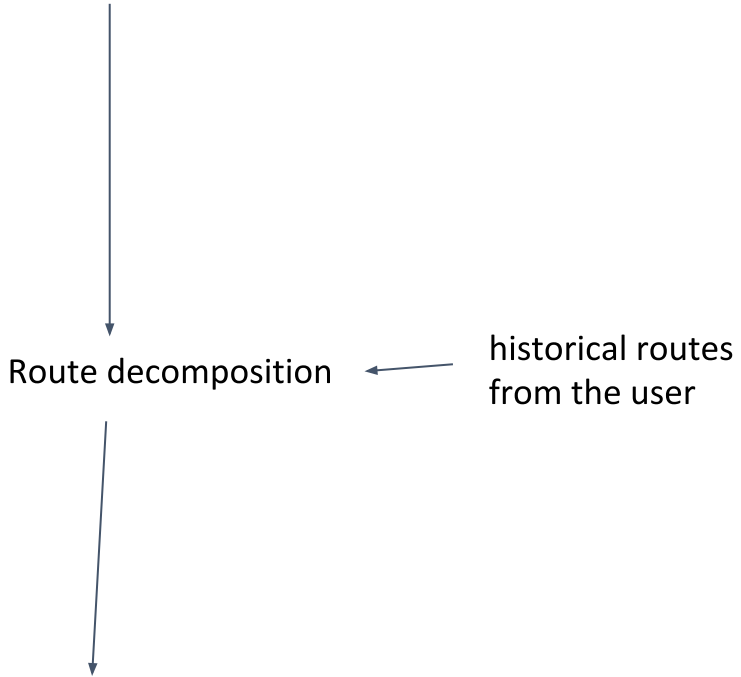
- Filter tweets for protest or disruption-related terms
 - Positive keywords
 - Negative/Stopwords keywords
 - Labeling datasets for machine learning
 - Prediction of protest likelihood based on tweet data

```
positives = df[(~df.text.str.contains('^do )|^how )|^will )|^shall )'
              ...\b)', regex=True))
              &
              (df.text.str.contains('\bforc to\b)|\bsay\b)|\bwarn\b)'
              ...
              '|(\bworri\b)|(\bhit\b)|(\btarget\b)|(\breport\b)'
              ..take down\b)', regex=True))]

negatives = ...
data = get_dataset(positives, negatives)
X_train, X_test, y_train, y_test = prepare_data_for_training(data)
pipeline = build_pipeline()
print("Training classification model")
model = train_model((X_train, y_train), pipeline, get_gridsearch_params())
print("Evaluating trained model")
evaluate_model(model, (X_test, y_test))
```

Abnormal route detection

Input daily route for a user



(route1, likelihood)
(route2, likelihood)
...

Maximum Likelihood route
Comparative statistics among routes

- Filter daily travel routes for abnormal activity
- Assign a probability-based measure on how likely the route is, compared to prior routes, of a user

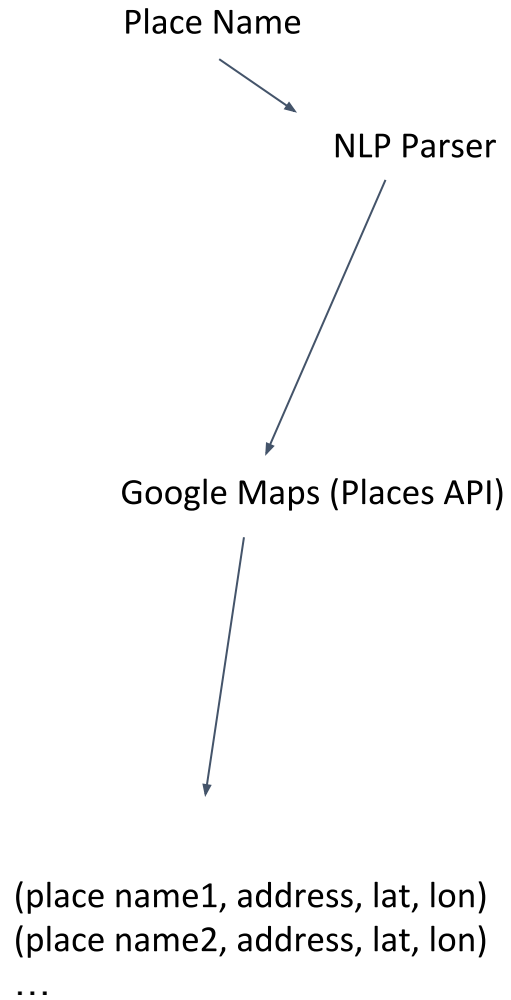
```
routes # list of all routes by (location, time)

vn = []
for route in routes:
    prob_route = solve_pde(route, routes)
    vn += [prob_route]

vnn = max_entropy(np.array(vn))
most_unlikely_route = vn[argmin(vnn)]

for ix in argmin(vnn):
    print vnn[ix], route[ix] # route entropy and route
```

Approx. Place name -> (Lat, Lon)



- News articles rarely have a (lat, lon) associated with place names. Twitter only provides (lat, lon) if geolocation is turned on and confirmed.
- Extract a (lat, lon) for a given place name, given variants and misspellings.

NLP

```
self.types = ['LOC', 'GPE', 'FAC']  
self.nlp = spacy.load('en_core_web_sm')
```

Google Maps Places

```
def get_complete_address(self, address, session_token=""):  
    places = self.gmaps.places_autocomplete(address, session_token)  
    return places
```

```
def get_address_coordinates(self, place, session_token=""):  
    results = self.gmaps.places(place, session_token)["results"]
```

run program on 2 sample place names

```
lm = LocationMining()  
print(lm.get_address_coordinates('Tbilisi'))  
print(lm.get_address_coordinates('vladjivostok')) # misspelled
```

```
{'formatted_address': 'Tbilisi, Georgia', 'geometry': {'location': {'lat':  
41.7151377, 'lng': 44.827096}, 'viewport': {'northeast': {'lat':  
41.8438937, 'lng': 45.0176811}, 'southwest': {'lat': 41.6210248, 'lng':  
44.6600246}}}, 'types': ['locality', 'political']}  
{'formatted_address': 'Vladivostok, Primorsky Krai, Russia', 'geometry':  
{ 'location': {'lat': 43.1198091, 'lng': 131.8869243}, 'viewport':  
{ 'northeast': {'lat': 43.289988, 'lng': 132.2311072}, 'southwest': {'lat':  
43.057424, 'lng': 131.840567}}}, 'types': ['locality', 'political']}
```

Raw Text ->
(Lat, Lon)
Program Demo

```
msyang(~/tmp/natural-language-processing.orig/news_nlp/utils/location_resolution)
```

```
jobs:4 $ python location_mining2.py "Vladivostok"  
Invalid -W option ignored: invalid module name: 'yaml'  
Input: raw_text: Vladivostok  
Output: location: {u'lat': 43.1198091, u'lng': 131.8869243}  
Output: full_address: Vladivostok, Primorsky Krai, Russia  
Output: type: [u'locality', u'political']
```

```
msyang(~/tmp/natural-language-processing.orig/news_nlp/utils/location_resolution)
```

```
jobs:4 $ python location_mining2.py "bolotnaya"  
Invalid -W option ignored: invalid module name: 'yaml'  
Input: raw_text: bolotnaya  
Output: location: {u'lat': 55.7456185, u'lng': 37.6169731}  
Output: full_address: Bolotnaya Ploshchad', Moskva, Russia  
Output: type: [u'route']
```

```
msyang(~/tmp/natural-language-processing.orig/news_nlp/utils/location_resolution)
```

```
jobs:4 $ python location_mining2.py "New York, NY"  
Invalid -W option ignored: invalid module name: 'yaml'  
Input: raw_text: New York, NY  
Output: location: {u'lat': 40.7127753, u'lng': -74.0059728}  
Output: full_address: New York, NY, USA  
Output: type: [u'locality', u'political']
```

```
msyang(~/tmp/natural-language-processing.orig/news_nlp/utils/location_resolution)
```

```
jobs:4 $ vi location_mining2.py
```

```
msyang(~/tmp/natural-language-processing.orig/news_nlp/utils/location_resolution)
```

```
jobs:4 $ python location_mining2.py "Cuauhtémoc Monument"  
Invalid -W option ignored: invalid module name: 'yaml'  
Input: raw_text: Cuauhtémoc Monument  
Output: location: {u'lat': 19.431085, u'lng': -99.1590175}  
Output: full_address: Av. Paseo de la Reforma S/N, Juárez, 06600 Ciudad de México, CDMX, Mexico  
Output: type: [u'point_of_interest', u'establishment']
```